

Agama Finance

Technical Architecture

Confidential Credit on Horizen



ERC-4626

UltraHonk

Horizen

PureFi

USDC.e

Vela

Contents

1. Summary

- 1.1 The Question This Document Answers
- 1.2 Design Principle
- 1.3 Proven, Observed, Trusted
- 1.4 Definitions

2. Platform Assessment

- 2.1 Horizen Chain, Measured
- 2.2 Vela, Measured
- 2.3 zkVerify
- 2.4 Where the Phase Boundary Falls
- 2.5 What We Exercised, and With What Result

3. Architecture Overview

- 3.1 System Diagram
- 3.2 Components
- 3.3 The Disclosure Cycle
- 3.4 The Loan Leaf
- 3.5 The Public Risk Surface

4. The Attestation Circuit

- 4.1 Statement Proven
- 4.2 What It Does Not Prove
- 4.3 Measured Performance

- 5. Contracts
 - 5.1 CreditDisclosureRegistry
 - 5.2 AgamaCreditVault
 - 5.3 IRiskSurface as a Horizen Primitive
 - 5.4 Compliance
- 6. Liquidity and the Exit Queue
- 7. Trust Model and Known Limits
- 8. Verification Results
 - 8.1 Live on Horizen Testnet
 - 8.2 Test Suite
- 9. Phase 2, Vela Integration

Status of this work. This is an early build, started in response to your review and roughly two weeks old. It is a prototype meant to test whether the approach holds, not a product. The book behind every figure is synthetic, nothing has run on mainnet, there is no originator and no depositor, the attestation circuit is capped at 64 positions, and none of it has been audited. What follows should be read as a design we have taken far enough to trust the shape of, with most of the work still ahead.

WHERE THE EVIDENCE FOR THIS DOCUMENT LIVES

Every claim below is checkable without taking our word for it. The repository accompanying this application is laid out so that each section here has a folder behind it.

Folder	Contents	Sections it backs
zk/	book_attest and book_delta Noir circuits, the scenario generator, and the proving script that also runs the backdating attack on every build	4 , 4.4 , 5.1
tee/	The full Vela stack we ran locally, the confidential lifecycle we executed on it, and the five findings that took it off our critical path	2.2 , 9
contracts/	Registry, vault, IRiskSurface, both generated verifiers, and thirty tests: fork, adversarial and stateful invariant	5 , 6 , 8.2
deployment/	The live Horizen testnet deployment: scripts, addresses, every transaction hash, and a reader that dumps the public surface using only <code>eth_call</code>	8.1

The contracts are source-verified on the Horizen explorer, so the code running at the addresses in [Section 8.1](#) can be read directly rather than trusted.

1. Summary

1.1 The Question This Document Answers

Horizen's review of our first submission made one observation that this entire architecture exists to answer:

“If the issuer signs a clean book and the loans are already bad, the TEE and the ZK proof still pass.”

That is correct, and no amount of additional cryptography changes it. A trusted execution environment attests that code ran on some data. A zero-knowledge proof attests that numbers follow from some commitment. Neither one reaches the world. A design that answers this objection by adding more cryptography is answering a different objection.

The design below therefore does not try to prove that loans are good. It makes the claim that they are good **falsifiable in public, on a fixed clock, without naming a single borrower**. A clean signature over a bad book still produces a valid proof. It then fails the cash calendar it published a month earlier, mechanically, with nobody's cooperation required.

1.2 Design Principle

Confidentiality covers identity. It does not cover risk.

Hiding a borrower's name is a legitimate commercial requirement and we defend it. Hiding the shape of an exposure is not a privacy feature: it is an information advantage held over the person who takes the loss. Our first submission published NAV and vault size, which let a depositor check arithmetic and nothing else. Every risk-bearing property of the book is now public, in aggregate, proven against a commitment, and machine readable.

1.3 Proven, Observed, Trusted

Every claim this system makes falls into exactly one of three categories, and the document is explicit about which:

Category	Mechanism	Examples
Proven	Zero-knowledge proof against an on-chain Merkle commitment, verified on Horizen	Outstanding principal, maturity ladder, delinquency buckets, concentration, forward cash calendar
Observed	On-chain state, no assertion by any party	Cash collected, cash disbursed, redemption queue depth, first-loss remaining, who requested book access and when
Trusted	Residual, bounded, and named in Section 7	That the committed leaves describe real loans; that the verification agent is honest

The third row is the honest one. It is small, it is named, and it is bounded by economics rather than by a promise: a book may never claim more principal than the chain watched leave the vault, so inventing exposure costs real, publicly visible cash.

1.4 Definitions

Term	Definition
Risk surface	The set of aggregate credit properties published for a book: count, principal, maturity ladder, forward cash calendar, delinquency buckets, largest single-obligor exposure.
Book root	Merkle root over the loan leaves. Committed on-chain before the positions it covers are funded.
Forward cash calendar	Contractual cash due over the 30 days following a valuation date, published <i>before</i> that period begins.
Performance ratio	Realised collections divided by the calendar published in advance for the same period.
Impairment schedule	A fixed public write-down per delinquency bucket, applied mechanically. The originator has no discretion over its own mark.
First loss	Originator capital escrowed in the vault, junior to every depositor, consumed before any depositor loss.
Verification agent	An independent party with full book access. Its access requests are on-chain transactions.
Vela	Horizen's TEE-based confidential coprocessor. Closed beta, see Section 2.2 .

2. Platform Assessment

Everything in this section was measured against live endpoints on 15 September 2026, not taken from documentation. The measurements are reproducible from the repository accompanying this document.

2.1 Horizen Chain, Measured

Property	Measured value
Chain ID	26514, eth_chainId returns 0x6792
Stack	OP Stack rollup on Caldera, settles to Base, Base settles to Ethereum
Block time	1.00 s, averaged over 50,000 blocks
Gas price	1,000,764 wei, about 0.001 gwei. Gas token is ETH
Block gas limit	30,000,000
Chain activity	307 transactions across the last 300 blocks
USDC.e total supply	3,166.65 USDC
PureFi Verifier	0x681Edd4906e2a0a277E2A6c394A4595f83e1329c, 2,008 bytes of code, callable
Stork Oracle	0xacC0a0cF13571d30B4b8637996F5D6D774d4fd62, deployed

Two of these rows shape the design.

The stablecoin supply. Horizen mainnet has roughly three thousand dollars of USDC.e on it. There is nothing to lend and nothing to lend against. A credit product here is not competing for existing liquidity, it is the reason liquidity would arrive. That is the strategic case for funding a money market primitive rather than an application.

The gas price. At 0.001 gwei, verifying a zero-knowledge proof directly on Horizen costs about two thirds of a US cent (Section 4.3). Daily attestation of a full credit book costs under 2.40 USD a year. This removes the usual reason to attest monthly, and it changes what the product can promise: disclosure becomes a continuous signal rather than a periodic report.

2.2 Vela, Measured

We ran Vela rather than reading about it. The full v0.2.0 stack from [HorizenOfficial/vela-starterkit](#) was brought up locally, nine containers, and a complete confidential lifecycle executed using the reference application and wallet from [vela-nova v0.2.0](#). The contracts we read to establish what leaks are in [HorizenOfficial/vela](#). Every command, and the five findings it produced, are in `tee/README.md` :

- WASM application deployed into the enclave, application ID `7335106231336828932` .
- P-521 user key registered on-chain via an `ASSOCIATEKEY` request.
- 1 ETH deposited, private balance readable only by the key holder.
- Deanonymisation authority granted through `DefaultAuthority.addAllowedAuthority` , emitting a public event.
- Deanonymisation report requested on-chain, generated inside the enclave, encrypted to the authority's P-521 key, downloaded and decrypted successfully.

The last step is the one that matters to this architecture: a designated authority, and only a designated authority, can pull private state, and **the act of pulling it is an on-chain transaction that everyone can see**. Requests from an unauthorised address revert with `AuthorityNotAllowed` , which we confirmed.

Where Vela actually is, because two Horizen pages disagree

[docs.horizen.io/vela/limitations](#) states that Vela “is not yet deployed to any testnet or mainnet environment” and that the development environment “uses an emulated Trusted Execution Environment rather than hardware enclaves”. The local stack matches that description: `TEE_NO_ATTESTATION=true` , and all three enclave keys sit in plaintext in the shipped `.env.dev` .

[The product's own page](#) says something different, and more favourable. Vela launched publicly on 16 March 2026 and is “**live on Base Sepolia for early-access developers**”, with a first roadmap phase listed in this order:

Step	Horizen Labs' own wording	Where Horizen is
1	“Live on Base Sepolia for early-access developers”	shipped by them
2	“Promote to Base mainnet”	ahead of them
3	“Deploy to Horizen testnet and mainnet”	ahead of them, and what we wait on

Alongside it: “Vela is open to developers now. Tell us what you're building and we'll get you into an environment.”

The accurate statement is narrower than the limitations page implies. Vela is deployed, on Base Sepolia, to developers who ask for access. What does not exist yet is Vela *on Horizen*, and that is step three of Horizen's own first phase rather than an open question. The emulated enclave and the plaintext keys are properties of the local development stack, not of the hosted environment.

We could not confirm the Base Sepolia deployment on-chain, because no addresses are published for it, and the disagreement between the two pages is worth putting to them directly. What follows from it for this design is narrow: Vela stays off the Phase 1

critical path, because a dependency that is live on a different chain than ours is still one we cannot ship against today. It does not reduce what Phase 2 is worth, and it makes Phase 2 a bet on a roadmap item Horizen has already committed to in public.

What Vela makes public, by design

This is a design input rather than a criticism, and it happens to suit a credit vault. The `ProcessorEndpoint` interface and the local chain agree:

Public on-chain	Private inside the enclave
Sender address of every request	Request payload
assetAmount on every deposit, a plain calldata field	Internal ledger state
Withdrawal recipient, token and amount	Per-user balances
Request timing, success or failure	UserEvent contents, P-521 encrypted
State root after every transition	—
Every authority grant, revocation and report request	—

Vela's perimeter is public and its interior is private. Money entering and leaving is fully visible with amounts and addresses; only the internal bookkeeping is hidden. For confidential credit this is the correct shape: we want the cash rail public and the borrower list private. Phase 2 builds on it rather than against it.

Other constraints recorded

- One WASM application per environment in v0.2.0.
- TinyGo only. No `reflect`, no `net`, no `math/big`, and strict determinism: no `time.Now()`, no randomness.
- Execution is stateless per call. The whole application state is serialised in and out of every invocation, which caps practical book size.
- A counterparty must register a P-521 key before it can receive anything. Our first private transfer failed with `no Secp521r1_PubKey found` until the receiver registered. Every borrower and LP must be onboarded with a key.
- `AuthorityRegistry.setAppAuthorityContract` lets an application supply its own authority checker, so the rule for who may look can itself be a smart contract. Phase 2 uses this.

2.3 zkVerify

[zkVerify](#) is live as its own L1 and its EVM verification contracts are deployed on Base, Sepolia, Base Sepolia, Arbitrum Sepolia, Optimism Sepolia and EDU Chain Testnet. **Not on Horizen L3.** Consuming a zkVerify aggregation from a Horizen contract would require a relay that Horizen does not currently run.

More decisively, zkVerify exists to solve a cost problem: verifying a proof on Ethereum L1 costs 200,000 to 300,000 gas at L1 gas prices. On Horizen at 0.001 gwei that argument does not apply. We verify directly on Horizen and treat zkVerify as an optional later optimisation for aggregating proofs across many originators, not as a dependency.

2.4 Where the Phase Boundary Falls

Component	What we found	Where we are
Horizen mainnet, EVM, ERC-4626	Live, 1 s blocks, effectively free gas	in use
USDC.e	Live, 3,166 USDC outstanding	in use
PureFi AML and KYC	Live, verified callable	in use
On-chain ZK verification	2,178,509 gas on a mainnet fork, 2,364,745 live on testnet	in use
Stork oracle	Live	not needed yet
zkVerify	Live, not deployed on Horizen L3	set aside
Vela TEE	Live on Base Sepolia for early-access developers, not yet on Horizen	waiting on step 3

2.5 What We Exercised, and With What Result

Our Builder Fund application, submitted on 4 September 2026, named one problem as genuinely unsolved. Section 3 of that application said it in these words:

“What remains genuinely unsolved is detecting an issuer that signs accurate-looking but false state. That's an attestation and audit problem, not a cryptographic one, and we don't pretend otherwise.”

The review that came back put the same problem more sharply, and this document is the work we did about it. Before designing anything we went through Horizen's own stack component by component, ran what could be run, and let the findings decide the design rather than the other way round.

What we exercised of the Horizen stack

Each box is a Horizen component. The badge is how far we have taken it in roughly two weeks of work, not how far it could go.

RUN LOCALLY, EARLY

Vela

TEE coprocessor, v0.2.0 starter kit, locally

Nine containers, full lifecycle executed:
WASM app into the enclave
P-521 key registered on-chain
confidential deposit, balance read back
authority granted, report decrypted

What we found, and it is not what the docs say
live on Base Sepolia, not yet on Horizen

FIRST CIRCUITS WORKING

Zero knowledge

Noir + UltraHonk, verified on Horizen

Two circuits, written not described:
book_attest, the risk surface, 66k opcodes
book_delta, honest revisions, 6.9k opcodes
3.0 s and 1.2 s to prove
verified on-chain, 2,364,745 gas

Finding on zkVerify
live, but its EVM contracts are not on Horizen L3. At 0.001 gwei, direct is free.

TESTNET ONLY

Horizen Chain

testnet 2651420, mainnet 26514

Live, and source-verified on the explorer:
full disclosure cycle executed on-chain
two surfaces and one revision verified
four-party queue settled at one ratio
ten attacks mined as reverts

Operational note
the testnet sequencer stalled twenty minutes mid-deployment, then caught up.

WIRED IN, UNTESTED LIVE

PureFi

AML, live on Horizen mainnet

Confirmed callable on-chain and wired into the deposit path.

A failed screen reverts the whole transaction, so a non-compliant deposit never exists as state.

Verifier proxy
0x681Ed49...83e1329c

What the evaluation has already changed in the design

Vela waits, because it is live on Base Sepolia and not yet on Horizen. That is step three of Horizen Labs' own first phase, not an open question.
zkVerify becomes optional rather than a dependency, because on this chain a proof verifies directly for about two thirds of a US cent.

The question our own application left open, in its own words

"What remains genuinely unsolved is detecting an issuer that signs accurate-looking but false state.
That's an attestation and audit problem, not a cryptographic one, and we don't pretend otherwise."

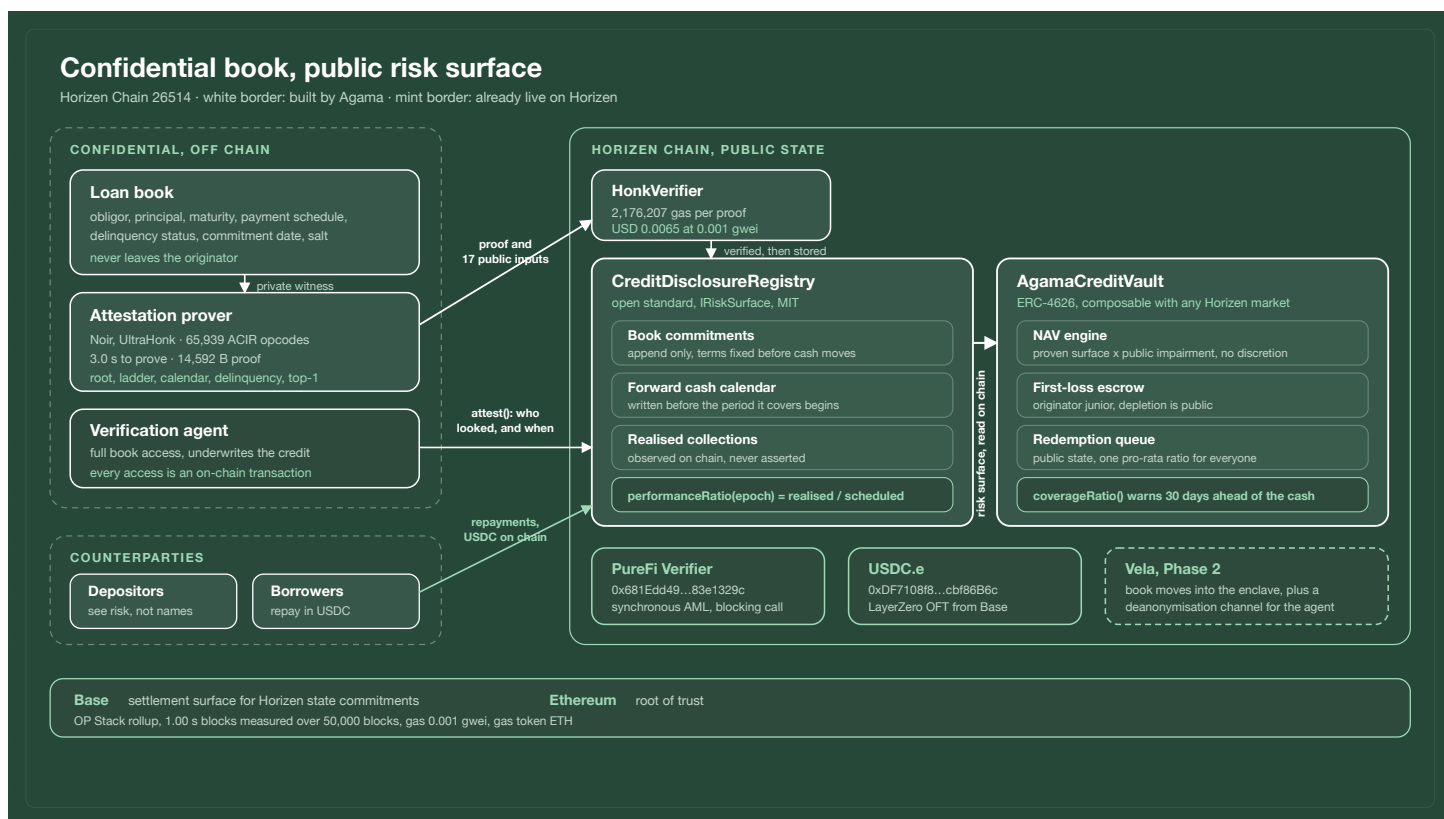
Builder Fund application, section 3, 4 September 2026

Component	How far we have taken it so far	Where to check it
Vela	Full v0.2.0 stack run locally, complete confidential lifecycle executed	tee/README.md, every command and the five findings
Zero knowledge	Two circuits written, proven, and verified on-chain by Horizen	zk/book_attest, zk/book_delta, rebuild with zk/prove.sh
Horizen Chain	Deployed on testnet 2651420, source-verified on the explorer	deployment/, every transaction hash. Read the live state with deployment/read-state.sh
PureFi	Confirmed callable on mainnet and wired into the deposit path	0x681Edd4906e2a0a277E2A6c394A4595f83e1329c
zkVerify	Evaluated and set aside, with a reason	Section 2.3

Two of those findings changed the architecture rather than confirming it. Vela left the critical path, because a hardware guarantee that is not deployed anywhere cannot carry a mainnet product. zkVerify became optional rather than a dependency, because on this chain a proof verifies directly for about two thirds of a US cent. Neither conclusion is one we expected going in.

3. Architecture Overview

3.1 System Diagram



3.2 Components

Off-chain, confidential

- **Loan book.** Obligor identifier, principal, maturity, next payment date and amount, delinquency status, commitment timestamp, blinding salt. Never leaves the originator.
- **Attestation prover.** Noir circuit compiled to UltraHonk. Takes the book as a private witness and emits a proof plus seventeen public field elements.
- **Verification agent.** An independent party with full book access that underwrites the credit properly. Every access is recorded on-chain.

On Horizen mainnet, public

- **HonkVerifier.** Generated from the circuit's verification key. 22,063 bytes of runtime code against the 24,576 byte limit.
- **CreditDisclosureRegistry.** Book commitments, proven risk surfaces, the forward cash calendar, realised collections, and the verification-agent attestation log. MIT licensed, no dependency on Agama's vault.
- **AgamaCreditVault.** ERC-4626 over USDC.e. NAV engine, first-loss escrow, public redemption queue.
- **PureFi Verifier.** Already deployed by Horizen. Synchronous AML enforcement as a blocking call.

3.3 The Disclosure Cycle

Ordering is the substance of this design, not an implementation detail. Each step is constrained to happen before the one that follows, and the contracts enforce it:

#	Step	Why it must happen here
1	<code>commitBook(root)</code>	Terms and payment schedules are fixed on-chain <i>before</i> anyone knows the outcome. The commitment is what a later proof is measured against.
2	<code>deploy(to, amount, root)</code>	Cash moves only against an already-committed book. The transfer is public, and it raises the ceiling on how much principal any future surface may claim.
3	<code>publishSurface(proof, inputs)</code>	The proven aggregates are stored, and the forward cash calendar is written to the epoch it covers. Writing it later than the period would be worthless, so the contract writes it first.
4	<code>recordCollection(amount)</code>	Borrower repayments arrive as on-chain stablecoin transfers. Realised cash is observed, never reported.
5	<code>performanceRatio(epoch)</code>	Realised against scheduled. This is the number a signed-but-bad book cannot survive.
6	<code>attest(root, asOf, findings)</code>	The verification agent records that it looked and what it concluded. The depositor reads this, and reads its absence.

3.4 The Loan Leaf

Each position is a leaf in a Merkle tree over 64 slots per attestation batch. The leaf hash commits to every field, so no field can change without changing the root:

```
pub struct Loan {
  obligor: Field,      // salted obligor id, stable across attestations
  principal: u64,      // outstanding principal, 6dp stablecoin units
  maturity_ts: u64,    // final maturity
  due_ts: u64,         // next scheduled payment date
  due_amount: u64,     // amount contractually due at due_ts
  status: u8,          // 0 current | 1 1-30dpd | 2 31-60 | 3 61-90 | 4 90+
  committed_ts: u64,   // when this leaf was first committed on-chain
  salt: Field,         // blinding factor
}
```

The `obligor` field is a salted identifier rather than a name. It is stable across attestations, which is what lets the circuit aggregate exposure per borrower and prove a concentration figure without revealing who anyone is. The `salt` prevents a dictionary attack on a leaf whose other fields an observer might guess.

3.5 The Public Risk Surface

Each attestation publishes the following, all proven against the committed root:

Field	Meaning	Reference book
<code>positions</code>	Active position count	22
<code>totalPrincipal</code>	Outstanding principal	4,760,000 USDC
<code>ladder[5]</code>	Principal maturing within 30, 90, 180, 365 days and beyond	190k / 660k / 995k / 1,530k / 1,385k
<code>dueNext30d</code>	Contractual cash due over the next 30 days	33,487.50 USDC
<code>delinquent[5]</code>	Principal by delinquency bucket	4,215k current, 435k 1-30dpd, 110k 90+
<code>top1</code>	Largest single-obligor exposure	1,050,000 USDC, 22.05%
<code>maxCommittedTs</code>	Newest per-leaf commitment timestamp	anti-backdating watermark

A depositor reading that row knows they are buying a 22-position book with a 22 percent top-name concentration, 2.31 percent already ninety days past due, and 190,000 USDC maturing inside the month. They know the risk shape they are taking. They still cannot name one borrower, and they never will.

4. The Attestation Circuit

Source in `zk/book_attest`, written in [Noir](#) and proven with [Barretenberg](#)'s UltraHonk backend. `zk/prove.sh` rebuilds every proof quoted in this document from scratch, and fails the build if the backdating attack in [Section 4.4](#) ever produces a valid proof. The generated verifier is deployed and source-verified on Horizen at [0xb95B3603...](#), so a proof in this document can be checked against the contract that accepted it.

4.1 Statement Proven

Given a private array of 64 loan leaves and a public valuation date, the circuit proves all of the following simultaneously:

- The published Merkle root is the root of exactly these leaves.
- Position count and outstanding principal are the honest sums over the active leaves.
- The maturity ladder buckets every position correctly relative to the valuation date.
- The forward cash calendar is the sum of `due_amount` over every position whose next payment falls in the 30 days after the valuation date.
- Delinquency buckets partition the principal by status, and no status is out of range.
- The concentration figure is the true maximum exposure aggregated per obligor, computed over all 4,096 obligor pairs.
- **No leaf claims a commitment timestamp later than the valuation date it is published under.**

The last constraint is the circuit's contribution to the honesty problem. Without it, a defaulted position could be inserted after the fact and backdated into a clean history.

4.2 What It Does Not Prove

The circuit does not prove that the loans are good, and no circuit can. It proves that whatever the originator committed to, the published numbers describe it exactly, and that the commitment predates the outcome. Loan quality is established afterwards, in public, by the forward cash calendar the circuit emits and the collections that either match it or do not.

This is stated in the circuit's own source comment rather than only in documentation, because the distinction is the product.

4.3 Measured Performance

Metric	Value
Toolchain	Noir 1.0.0-beta.13, UltraHonk, keccak oracle
ACIR opcodes	65,939
Circuit size	2^{18}
Proving time	3.0 s on a laptop, 14 threads
Proof size	14,592 bytes
Public inputs	17 field elements, 544 bytes
Verifier runtime size	22,063 bytes, 2,513 bytes below the contract limit

On-chain cost, measured on a fork of Horizen mainnet

Operation	Gas	ETH	USD at 3,000
commitBook	98,485	0.0000000986	0.00030
deposit, ERC-4626	127,087	0.0000001272	0.00038
publishSurface, full verification	2,178,509	0.0000021802	0.0065
settle, two queue tickets	85,100	0.0000000852	0.00026

One caveat worth recording: the verifier sits 2,513 bytes below the 24,576 byte contract size limit. Materially growing the public input set will require splitting the verifier or moving to a proof system with a smaller one.

4.4 The Revision Circuit

The attestation circuit proves that a surface honestly describes whatever book was committed. It cannot, on its own, prove that a *revision* of the book is an honest evolution of the previous one. Without that, an originator could publish a second root containing a loan that had already defaulted, stamped with an old origination date, and every surface over it would still verify.

book_delta closes this. Its central design decision is that a position holds a permanent slot for the life of the book. Slots are never reused, so a repaid loan sits at zero principal rather than freeing its slot for a replacement, and matching two books becomes a linear walk rather than a search.

Operation	Rule proven
Carry a position	Obligor, maturity, origination date and salt are unchanged. Principal may only amortise down, never re-advance. The next payment date may only move forward.
Revise an amount due	The amount due at an unchanged payment date may not be lowered, so the bar a position is judged against cannot be moved once set.
Change status	Free in both directions. Deterioration has to be expressible; that is the point of a revision.
Open a position	Only into a slot that was empty, and only with an origination date at or after the moment the previous root reached the chain.
Use an empty slot	Nothing may hide in it: principal, obligor and timestamp must all be zero.

The fourth row is the one that matters. The registry supplies the previous root's on-chain commitment timestamp as a public input and refuses any proof that disagrees with what the chain recorded, so the circuit is always measured against a date the originator did not choose.

Metric	Value
ACIR opcodes	6,906, roughly a tenth of the attestation circuit
Proving time	1.2 s
Public inputs	4: old root, previous commitment time, valuation date, new root
On-chain verification	2,222,038 gas, measured on Horizen testnet

Demonstrated, not asserted. `zk/prove.sh` builds an `evil` scenario in which a defaulted 900,000 USDC position is dropped into a free slot with an origination date 200 days before the previous commitment. Witness generation fails with `Assertion failed: new position backdated`. The build script treats a successful proof of that scenario as a build failure.

5. Contracts

5.1 CreditDisclosureRegistry

The registry holds three things that a confidential credit product normally keeps in a side letter: the commitment history, the calendar the book promised, and the record of who has been allowed to look at it.

Anti-backdating, at four levels

1. **In the attestation circuit.** No leaf may claim a commitment timestamp later than the valuation date it is published under.
2. **In the revision circuit.** Every new version of the book is proven to descend from the previous one by permitted operations only. See [Section 4.4](#).
3. **In the registry.** A surface is rejected unless its root was committed earlier, and unless the newest leaf commitment predates the moment that root reached the chain.

```

if (committedAt == 0)            revert RootNotCommitted();
if (asOf < committedAt)         revert AsOfBeforeCommit();
if (maxCommittedTs > committedAt) revert LeafBackdated();

```

4. **In economics.** A book may never claim more principal than the chain watched leave the vault:

```

if (claimedPrincipal > deployedCumulative) revert PrincipalExceedsDisbursed();

```

Disbursement is a public transfer, so inventing exposure costs real, visible cash. This is the binding constraint, and it is the reason the first two are enough in practice.

Mechanical impairment

The write-down applied to each delinquency bucket is fixed at deployment and public:

```

uint16[5] public IMPAIRMENT_BPS = [0, 1000, 3000, 6000, 10000];

```

Carrying value is `totalPrincipal - impairedPrincipal()`, a pure function of the proven surface and this constant. Nobody signs a NAV. In the reference scenario, impairment moves from 153,500 to 246,500 USDC between two attestations on the same positions with no human marking anything.

Verification access as public state

`attest(root, asOf, findingsHash)` records who looked, when, at which book, and what they concluded.
`verificationAge()` returns the seconds since anyone last did. A depositor cannot read the book. A depositor can read the fact that an independent party with full access is reviewing it on a fixed cadence, and can read the silence when it stops.

5.2 AgamaCreditVault

ERC-4626 over USDC.e, so shares compose with anything on Horizen. Three behaviours differ from a standard vault, all deliberately.

NAV is computed, not asserted

```

function totalAssets() public view override returns (uint256) {
    uint256 principal = registry.surface().totalPrincipal;
    uint256 impaired  = registry.impairedPrincipal();
    uint256 absorbed  = impaired < firstLoss ? impaired : firstLoss;
    return buffer() + principal - impaired + absorbed;
}

```

Every term is either an on-chain balance or a proven public input. There is no path by which the originator marks its own book.

First loss is on-chain and junior

Originator capital is escrowed in the vault and consumed before any depositor loss. `firstLossRemaining()` is public, so its depletion is visible as it happens rather than at the moment it stops protecting anyone. In the reference scenario it is fully consumed at 246,500 USDC of impairment, and the share price then falls from 1.0000 to 0.9915 with no intervention.

Instant redemption is disabled

`maxWithdraw` and `maxRedeem` return zero. An instant exit for the fastest depositor is the mechanism that converts illiquidity into a run. See [Section 6](#).

5.3 IRiskSurface as a Horizen Primitive

Horizen's review asked for "a money market primitive we can diligence on-chain". We read that literally: a *contract* should be able to underwrite this vault, not only a human reading a document.

```
interface IRiskSurface {
  function surface()          external view returns (Surface memory);
  function impairedPrincipal() external view returns (uint256);
  function scheduledFor(uint64 epoch) external view returns (uint128);
  function realizedFor(uint64 epoch)  external view returns (uint128);
  function performanceRatio(uint64 epoch) external view returns (uint256);
  function concentration()          external view returns (uint256);
  function attestationAge()         external view returns (uint256);
}
```

A Horizen lending market can set an LTV on vault shares programmatically: refuse collateral whose `attestationAge()` exceeds a threshold, haircut by `concentration()`, liquidate on a `performanceRatio()` below a floor. That is on-chain diligence in the literal sense.

The interface and the reference registry are MIT licensed with no dependency on Agama's vault. We are proposing them as the disclosure standard for confidential credit on Horizen. If a second originator adopts the standard and Agama never earns a dollar from them, the grant has still done its work.

5.4 Compliance

PureFi is live on Horizen mainnet and the vault gates deposits on it. `depositWithCompliance` calls `validatePayload` as its first operation, so a failed AML check reverts the entire transaction and a non-compliant deposit never exists as state. Our fork test confirms both that the verifier is deployed and that an invalid payload reverts the deposit.

6. Liquidity and the Exit Queue

Horizen's review noted that weekly redemptions leave the depositor last to see a run. The weekly gate is gone, and it was the right thing to remove: a discretionary gate creates the run it is meant to prevent, because being early pays.

Property	Mechanism	Effect
The queue is public	<code>RedemptionRequested</code> events plus <code>queueDepth()</code> and per-ticket getters	No depositor learns about an exodus after the others. The queue is the signal.
Settlement is pro-rata	<code>settle()</code> computes one ratio for the whole open queue and is permissionless	Queueing first buys nothing, so there is no race to queue.
Shortfall is announced early	<code>coverageRatio() = (buffer + 30-day inflows + 30-day maturities) / queued</code>	A depositor sees the gap about a month before it binds.
No discretion exists	No role can choose who is paid or reorder the queue	There is nothing for a manager to decide under pressure.

In the reference scenario two LPs holding 3.3 M and 2.2 M of a 5.5 M vault queue one block apart. Both receive 13.45 percent of what they asked for. Pushed harder, with forty tickets spanning two orders of magnitude and each joining a block later than the last, every single one fills at 17.15 percent. `coverageRatio()` reads 17.51 percent, and it reads that before the cash is needed rather than after.

What this does not fix. Illiquidity does not disappear. A book of nine-month loans cannot pay everyone next week and no mechanism changes that. What changes is that the depositor sees the shortfall coming, sees it at the same moment as everyone else, and gains nothing from panicking.

Settlement processes the entire open queue in one transaction, bounded by `MAX_QUEUE = 256`. Exceeding that bound is a public condition rather than a discretionary gate. A full 256-ticket queue settles in 13,434,703 gas, inside Horizen's 30,000,000 block limit, which we measured rather than assumed.

The reserve floor

Nothing above helps if the originator can deploy the entire vault into the book. A deployment is refused unless it leaves liquid stablecoins worth at least `floorBps` of assets under management, 1,000 basis points at deployment.

The base is deliberately assets under management rather than the liquid balance. A floor measured on the buffer alone can be walked to nothing by repeated partial deployments: take ninety percent, then ninety percent of what is left, and so on. Measured on a base that does not move when cash shifts from the buffer into the book, slicing buys the originator nothing.

`test_ReserveFloorCannotBeDrained` attempts exactly that and fails to get below the floor.

The floor binds deployments, not redemptions. A queue is entitled to drain the buffer, that is what the buffer is for, and the originator simply cannot deploy again until the book is back above its floor.

7. Trust Model and Known Limits

This section exists because a confidential credit product that will not name its residual trust assumptions has not earned the confidentiality.

7.1 What remains trusted

Assumption	What could go wrong	What bounds it
Committed leaves describe real loans	An originator commits fictional positions	Claimed principal can never exceed cash the chain watched leave the vault. Fiction costs real money, publicly.
Repayments route on-chain	A servicer collects fiat and does not forward it	The forward calendar is missed within one period, in public. Eligibility for the vault requires an on-chain repayment rail.
The verification agent is honest	A captured auditor signs clean findings	Attestations are on-chain and dated. A silent agent is as visible as a lying one, and the appointment is a public role a Horizen-appointed party can hold.
Book revisions are honest deltas	A new root inserts a backdated bad loan	Proven, not trusted. The revision circuit refuses it, against a commitment timestamp read from the chain rather than supplied by the originator.

7.2 The gap we closed

An earlier draft of this document named a gap here: a book revision published a new root, and nothing proved that the new book was the old book plus permitted operations. An originator could in principle have inserted a position with an older commitment timestamp, bounded only by the disbursement ceiling.

That gap is closed by the revision circuit in [Section 4.4](#), which is deployed and verified on-chain rather than scheduled. The backdating attempt it refuses is built on every run of the proving script, and a proof that succeeds is treated as a build failure.

What remains open

The book is 64 slots per attestation batch. A larger originator needs either several registries or a recursive aggregation over batches, which is engineering rather than research, but it is not built. The verifier also sits 2,513 bytes below the contract size limit, so growing the disclosed field set is not free.

7.3 What we do not claim

Which confidentiality this design delivers, and which it does not

This document covers one of the three confidentiality requirements our Builder Fund application set out, and it is deliberate that it covers the third rather than the first.

Requirement	Status here	Why
Credit-level detail: borrowers, maturities, negotiated terms	Working prototype	Commitments plus zero-knowledge proofs, verified on Horizen. Needs nothing that is not already live.
Depositor positions and identities	Not started	The vault is ERC-4626, so share balances are public. Hiding them needs a confidential ledger, which is exactly what Vela provides and what we ran locally. It is Phase 2 and it is blocked on Vela shipping.
Pending redemptions, confidential until execution	Deliberately inverted for now	The review made the opposite point: a depositor should not be last to learn others are leaving. Both concerns are real and the resolution is in Section 9 , not in a choice between them.

The book behind every figure in this document is synthetic, generated by `zk/gen.py`. There is no real originator, no real borrower and no real collection behind the testnet numbers. What is demonstrated is that the machinery works and refuses what it says it refuses. Putting a real book behind it is milestone M1.

We also cannot make a depositor able to underwrite this credit, and we will not pretend otherwise. Someone with full access to the book still has to do that work. The claim is narrower: a depositor can verify the risk shape, watch performance as a falsifiable series on a clock the originator committed to in advance, see that an independent party with full access is looking on a fixed cadence, exit through a public queue with no first-mover advantage, and know that the originator loses first.

That is the side letter, moved on-chain, where a contract can read it.

8. Verification Results

8.1 Live on Horizen Testnet

The system described in this document is deployed and running on Horizen testnet, chain 2651420. Not a fork and not a simulation. The full disclosure cycle was executed on-chain: the book committed before any cash moved, two risk surfaces and one revision proof verified by the chain, a short collection recorded, and the redemption queue settled pro-rata.

Contract	Address
CreditDisclosureRegistry	0x6D7C4a153C47841fE0A75C3b0a5298E1a3c9229B
AgamaCreditVault	0x48db9A42098f8eEc6ff14D771D67cA57f3aB5651
HonkVerifier	0xb95B360324edbfd324e19196F6fF3B97E9EaA7dA
DeltaVerifier	0xA3C1e81Fdadb5bd098546A4B8FBDfe4169436cF4
Test stablecoin	0xd0c379C0cD56b62fd9c2fE8913aa261f8096164e

All five are source-verified on the Horizen explorer. The code running at those addresses can be read by anyone, and `deployment/read-state.sh` dumps the entire public surface using nothing but `eth_call`. Every transaction hash behind this section is in `deployment/deployment-testnet.md` and `deployment/act2-testnet.md`.

Anyone can read the following from the registry today, without asking us for anything:

Reading	Value
Positions, outstanding principal	22, 4,760,000 USDC
Maturity ladder, 30 / 90 / 180 / 365 / beyond	190k / 660k / 995k / 1,530k / 1,385k
Delinquent, current / 1-30 / 31-60 / 61-90 / 90+	3,835k / 540k / 275k / 0 / 110k
Largest single obligor	1,050,000 USDC, 22.06 percent
Impairment, mechanical	246,500 USDC
Carrying value	4,513,500 USDC
Epoch 0 scheduled	33,487.50 USDC
Epoch 0 realised	12,000.00 USDC
Epoch 0 performance	35.83 percent
First loss remaining	253,500 of 500,000 USDC
Book revisions committed	2, the second proven to descend from the first

The three bold rows are the whole argument reduced to three numbers. The book published what it was owed before the period began, the cash that arrived is an on-chain fact, and the ratio between them is a figure nobody had to agree to.

Operation, on testnet	Gas
<code>publishSurface</code> , risk surface with ZK verification	2,364,745
<code>commitRevision</code> , revision delta with ZK verification	2,222,038
<code>publishSurface</code> , deteriorated surface	2,192,403
<code>settle</code> , redemption queue	135,433
<code>commitBook</code>	115,022

The whole deployment plus the full cycle, including three on-chain UltraHonk verifications, cost 0.000000027 ETH.

Continuous disclosure is not a cost problem on this chain. It is a design choice we can simply make.

Independent verification and a real queue

The verification agent has recorded an attestation on-chain, so `attestationCount()` reads 1 and `verificationAge()` counts up from it. Four depositors then settled in one transaction, sizes spanning twenty to one, with the founding depositor first in line since a previous settlement and the other three joining afterwards:

Party	Queue position	Asked	Received	Fill
Founding LP	first, much earlier	2,010,000.00	462,068.97	22.989 %
LP1	later	100,000.00	22,988.51	22.989 %
LP2	later	200,000.00	45,977.01	22.989 %
LP3	later	300,000.00	68,965.52	22.989 %

Identical to three decimal places. The residual difference is integer rounding on a six-decimal asset, not an ordering advantage. This is the claim in [Section 6](#), executed on a public chain rather than asserted in a test.

Attacks refused on-chain, with hashes

Ten attacks were submitted to the chain as real transactions and mined as reverts. Each revert selector was recovered by replaying the call at the block before it was mined. Tampered public inputs are rejected inside the verifier; stale surfaces by monotonicity; replayed and mis-parented revisions by the descent check; a false previous-commitment date by the value the chain itself recorded; principal without disbursement by the ceiling; and the queue is not bypassable through `redeem`.

Two of those first carried the wrong label, and we corrected them. The registry checks in ascending order of cost, so a tampered surface with an old valuation date is rejected by monotonicity before the proof is examined, and a revision quoting the wrong parent never reaches the timestamp check. We reran both, shaped to pass the cheap guards and reach the intended one, and rewrote the table to say what happened rather than what we meant to show. Ordering cheap checks first is correct. Reporting a refusal as coming from a mechanism that never ran is not.

8.2 Test Suite

Thirty tests, in `contracts/test/`. Seven run the disclosure cycle against a live fork of Horizen mainnet, chain 26514. Seventeen attack the design rather than demonstrate it. Six are stateful invariants driven by randomised traffic.

Test	What it establishes
FullDisclosureCycle	Commit, fund, disburse, prove, mark. NAV derived from the proof, not asserted.
BadBookFailsItsOwnCalendar	The reviewer's case. The proof passes, the calendar is missed at 35.83 percent, first loss is consumed, the share price falls to 0.9915 mechanically.
RejectsUncommittedAndBackdatedBooks	A surface against an uncommitted root reverts. A surface predating its own commitment reverts.
QueueIsProRataAndPublic	Two LPs, one block apart, receive an identical fill ratio. Coverage is published before the shortfall binds.
VerifierAccessIsPublic	Attestation age is unbounded until someone looks, then zero, then grows visibly.
PureFiIsLiveOnHorizen	The live verifier has code and an invalid payload reverts the deposit.
ReportLiveChainCosts	Chain 26514, gas price 1,000,764 wei, USDC.e supply 3,166.

Attacks attempted, and refused

Attack	Outcome
Understate delinquency, overstate principal, or backdate the valuation date in the public inputs	Verification fails on-chain. The untouched proof still works, so the rejection is the tampering and not something unrelated.
Re-point a valid proof at a different committed book	Rejected.
Re-publish an older surface to walk back a disclosed deterioration	SurfaceNotMonotonic.
Fork a revision off an old root, or replay one	NotDescendedFromLatest.
Feed the circuit a false previous-commitment timestamp	WrongCommitTimestamp. The chain supplies that value, not the originator.
Claim principal no disbursement paid for	PrincipalExceedsDisbursed.
Salami-slice the vault below its reserve floor	Refused. The floor base does not move when cash shifts into the book.
ERC-4626 first-depositor inflation attack	Uneconomic. The attacker burns 10,000 USDC and the victim redeems 9,999 of their 10,000.
Gain from queueing early, 40 tickets spanning two orders of magnitude	Every ticket filled at 17.15 percent. Position buys nothing.
Bypass the queue through redeem or withdraw	Both revert. maxRedeem is zero by design.
Cancel someone else's ticket, or over-claim a partial fill	Refused. The remainder is returned to the unit.
Raise the disbursement ceiling without moving cash	NotVault.
Censor a disclosure by withholding it	Not possible. publishSurface is permissionless: anyone holding a valid proof can publish it.

Stateful invariants

Randomised deposits, redemption requests, cancellations, settlements and share transfers across 4,096 calls. Escrowed shares always equal the queue total; the queue total always equals the sum of open tickets one by one; queued shares never exceed supply; first-loss capital is never spent; no redemption creates value.

9. Phase 2, Vela Integration

Vela is the right destination and the wrong prerequisite. Phase 2 upgrades a system already in production rather than replacing a promise with another one.

Change	What it buys
Book state moves into the enclave	Aggregates are computed inside the TEE rather than proven off-chain, removing the proving step from the originator's critical path.
Verification agent becomes a Vela authority	Book access happens through DEANONYMIZATION, so the access request is itself an on-chain transaction rather than an off-chain arrangement we describe.
Custom IAuthorityChecker	The rule for who may look becomes a contract: for example, access only while an attestation is current, or only with a quorum of agents.
Trigger contract drives the vault	TRUSTPROCESS lets enclave state push directly into the vault's accounting without an external relayer.

9.1 The Confidentiality Phase 1 Does Not Provide

Phase 1 hides the book. It does not hide the depositor, and a vault whose share balances are public does not serve the institutional allocator our application was written for. That allocator's position size, entry timing and exits would all be readable, which is the specific objection that caps how much capital this design can hold.

The primitive that fixes it is one we have already run. Vela's reference application is a confidential balance ledger: we deposited into it, read a private balance back with our own key, and confirmed that no one else could. Moving the vault's share ledger inside the enclave is that same mechanism applied to a different accounting.

9.2 Resolving the Redemption Conflict Rather Than Choosing a Side

Our application said pending redemptions must stay confidential, because knowing a large LP has queued an exit before it settles is exploitable. The review said the opposite, that a weekly cycle leaves the depositor last to see a run. Both are correct, and they are only in conflict if the queue has to be all public or all private.

What the queue publishes	Which concern it answers
Total depth, total owed, coverage ratio, settlement ratio	Nobody is last to learn the vault is under pressure. This is Phase 1 behaviour and is live today.
Who queued, and for how much	Kept private in Phase 2, so a specific allocator's exit cannot be traded against. This needs the confidential ledger above.

The aggregate is the run signal, and the individual ticket is the exploitable detail. There is no reason for them to share a visibility setting, and Phase 1 publishes both only because Phase 1 has nowhere private to put the second one.

9.3 What Gates It

Phase 2 is gated on one specific item: “Deploy to Horizen testnet and mainnet”, step three of Vela's own first roadmap phase. Vela itself is already live on Base Sepolia for early-access developers, so this is not a bet on whether the technology arrives, only on when it reaches this chain.

It is worth stating the dependency plainly rather than as a hedge. We have run the Vela primitive that Phase 2 needs, locally and end to end, and confirmed it does what it claims. What we cannot yet do is run it here. An application waiting on step three is also the clearest argument for scheduling step three.